Here is the complete detailed process of everything we built from top to bottom.

Macsinfosolution Leave Management System

Complete Project Documentation

SECTION 1 — PROJECT OVERVIEW

What This Project Is

A complete Leave and Work From Home management system for Macsinfosolution company built with Angular 17 frontend and designed to connect to a PHP REST API backend.

Business Purpose

- Employees can apply for leave and WFH
- Report Managers approve or escalate requests
- Managers review escalated requests
- Admin has full visibility and control
- All actions trigger email notifications
- Documents can be attached to requests

SECTION 2 — TECHNOLOGY USED

Technology	Purpose	Version
Angular	Frontend framework	17
Angular Material	UI components	17
TypeScript	Programming language	5

SweetAlert2	Popup alerts	Latest
jsPDF	PDF generation	Latest
jspdf-autotable	PDF tables	Latest
Poppins	Font	Google Fonts
Node.js	Runtime	18+
npm	Package manager	9+

SECTION 3 — PROJECT FOLDER STRUCTURE

```
E:\Attendance App\attendance-frontend\
├── src\
│   ├── app\
│   │   ├── auth\
│   │   │   └── login\
│   │   │       ├── login.component.ts
│   │   │       ├── login.component.html
│   │   │       └── login.component.scss
│   │   │
│   │   ├── master-admin\
│   │   │   ├── dashboard\
│   │   │   ├── companies\
│   │   │   ├── roles\
│   │   │   ├── employees\
│   │   │   ├── leave-overview\
│   │   │   ├── wfh-overview\
│   │   │   └── holidays\
│   │   │
│   │   ├── report-manager\
│   │   │   ├── dashboard\
│   │   │   ├── leave-requests\
│   │   │   ├── my-employees\
│   │   │   ├── groups\
│   │   │   └── leave-overview\
│   │   │
│   │   ├── manager\
│   │   │   ├── dashboard\
│   │   │   └── leave-overview\
│   │   │
│   │   ├── employee\
```

```
│   │   │   ├── dashboard\
│   │   │   ├── apply-leave\
│   │   │   ├── apply-wfh\
│   │   │   ├── my-leaves\
│   │   │   └── my-wfh\
│   │   │
│   │   ├── shared\
│   │   │   ├── navbar\
│   │   │   ├── sidebar\
│   │   │   ├── profile\
│   │   │   └── layout.component.ts
│   │   │
│   │   └── core\
│   │       ├── services\
│   │       │   ├── auth.service.ts
│   │       │   ├── employee.service.ts
│   │       │   ├── company.service.ts
│   │       │   ├── leave.service.ts
│   │       │   ├── wfh.service.ts
│   │       │   ├── group.service.ts
│   │       │   ├── mock-data.service.ts
│   │       │   ├── pdf.service.ts
│   │       │   ├── notification.service.ts
│   │       │   └── email-notification.service.ts
│   │       ├── guards\
│   │       │   ├── auth.guard.ts
│   │       │   └── role.guard.ts
│   │       ├── interceptors\
│   │       │   └── auth.interceptor.ts
│   │       └── models\
│   │           ├── user.model.ts
│   │           ├── leave.model.ts
│   │           └── company.model.ts
│   │
│   ├── environments\
│   │   ├── environment.ts
│   │   └── environment.prod.ts
│   │
│   ├── assets\
│   │   └── logo.png
│   │
│   └── styles.scss
│
├── angular.json
```

```
├── package.json
└── proxy.conf.json
```

SECTION 4 — USER ROLES AND ACCESS

Role Hierarchy

```
Admin (Master Admin)
  ↓
Manager
  ↓
Report Manager (RM)
  ↓
Employee
```

What Each Role Can Do

Master Admin (admin/admin123)

- View all companies, employees, leaves, WFH
- Add/edit/delete companies
- Add/edit/delete employees
- Manage roles per company
- Approve/reject/escalate any leave
- Manage holidays
- View all reports and calendars

Manager (manager/mgr123)

- View all leaves in their company
- Approve/reject leaves escalated from RM
- Escalate to Admin
- Apply for their own leave (goes to Admin)
- View company calendar

Report Manager (rm/rm123)

- View leaves of their team employees only
- Approve/reject/escalate team leaves
- Create and manage groups
- Apply for their own leave (goes to Manager)
- View team calendar

Employee (employee/emp123)

- Apply for leave with optional document
- Apply for WFH
- View their own leave history
- Cancel pending leaves
- Reopen rejected leaves
- Download own leave details

SECTION 5 — LEAVE APPROVAL FLOW

When Employee Applies Leave

Employee submits leave
↓
Status: pending_rm
↓
RM sees the request
↓
RM Approves → status: rm_approved → END
RM Rejects → status: rm_rejected → END
RM Escalates → status: pending_manager
↓
Manager sees the request
↓
Manager Approves → status: manager_approved → END
Manager Rejects → status: manager_rejected → END
Manager Escalates → status: pending_admin
↓
Admin sees the request
↓
Admin Approves → status: admin_approved → END
Admin Rejects → status: admin_rejected → END

When RM Applies Leave

RM submits leave
↓
Status: pending_manager
↓
Manager approves/rejects/escalates

When Manager Applies Leave

Manager submits leave
↓
Status: pending_admin
↓
Admin approves/rejects

Status Labels

Status	Display
pending_rm	Pending RM
rm_approved	RM Approved
rm_rejected	RM Rejected
pending_manager	Pending Manager
manager_approved	Manager Approved
manager_rejected	Manager Rejected
pending_admin	Pending Admin
admin_approved	Admin Approved
admin_rejected	Admin Rejected

SECTION 6 — LEAVE TYPES

Type	Code	Max Days
Casual Leave	CL	12 days
Sick Leave	SL	6 days
Earned Leave	EL	15 days
Maternity Leave	ML	90 days
Paternity Leave	PTL	15 days

Compensation Leave	COMP	12 days
Unpaid Leave	UL	Unlimited

SECTION 7 — FEATURES BUILT

Login Page

- Username and password login
- Click to auto-fill credentials for demo
- Role-based redirect after login
- Idle timeout — auto logout after 5 minutes

Master Admin Dashboard

- Today's leave requests count
- Today's pending approvals count
- Employees on leave today
- WFH today count
- This month total, approved, pending, rejected
- Total companies, employees, active, inactive
- Quick action buttons

Companies Page

- Card view showing all companies
- Company code, address, employee count
- Click company to see all employees
- Add Employee button in company view
- Download All Leaves button (hidden for now)

Employees Page

- Table with search and company filter
- Role filter dependent on company selected
- Add employee with username, password, eye toggle
- Three dots menu with edit and deactivate

Leave Overview (Admin)

- List view with status filter

- Calendar view with daily counts
- Click date to see day detail panel
- Day panel shows total, approved, pending, rejected
- Holiday highlighting in calendar
- Weekend highlighting
- Approve/Reject/Escalate actions with reason

WFH Overview (Admin)

- Same structure as leave overview
- List and calendar view

Holiday Management

- List all holidays by category
- National, Festival, Company categories
- Add new holiday with name, date, type
- Delete holiday
- Holidays shown in all calendars

Report Manager Dashboard

- My employees count
- Pending requests count
- Approved this month count
- Total groups count
- Quick action buttons

Team Requests (RM)

- Leave tab and WFH tab
- Only shows team members leaves
- Approve/Reject/Escalate with reason
- Email simulation popup after action

Leave Overview (RM)

- List view with click to see detail panel
- Detail shows employee info, dates, reason
- Same week leave requests shown on right
- Approve/Reject buttons in detail panel

My Employees (RM)

- Shows only team members
- Download individual employee leaves

Groups (RM)

- Create group with member selection
- Only shows RM's team employees
- Add/remove members

Manager Dashboard

- Pending team requests
- My pending leaves
- Approved this month
- Rejected this month

Manager Leave Overview

- Company-wide leave view
- Click row to see detail panel
- Detail panel with Approve/Reject/Escalate
- Calendar view with daily counts

Employee Dashboard

- Leave balance summary
- Pending, approved, rejected counts

Apply Leave

- Leave type dropdown
- Date picker
- Reason text area
- Optional document upload (PDF, JPG, PNG max 5MB)
- Drag and drop support
- Shows past 30 days leave history on right side

Apply WFH

- WFH reason dropdown
- Date picker
- Description
- Optional document upload
- Shows WFH history this month below form

My Leaves

- All, Pending, Approved, Rejected tabs
- Summary cards showing counts
- Three dots menu on each row
- Cancel pending leave
- Reopen rejected leave
- Download individual leave PDF

My WFH

- Same structure as My Leaves
- All tabs working with filter

Profile Page

- Edit personal details only
- Name, phone, email, department, designation
- Change password optionally
- Company and role are read-only

SECTION 8 — COMMANDS TO RUN

Start the App

```
cd "E:\Attendance App\attendance-frontend"
ng serve --port 4201
```

Open in Browser

http://localhost:4201

If Port is Busy

```
ng serve --port 4202
```

Clear Cache and Restart

```
Remove-Item -Recurse -Force .angular
ng serve --port 4201
```

Install Packages

npm install

Build for Production

ng build

SECTION 9 — TEST CREDENTIALS

Role	Username	Password
Master Admin	admin	admin123
Report Manager	rm	rm123
Manager	manager	mgr123
Employee	employee	emp123

SECTION 10 — CONNECTING REAL API FROM LEAD

What to Ask Your Lead

Before starting ask your lead these questions:

1. What is the API base URL?
   Example: http://192.168.1.100/api

2. What are the login credentials for each role?
   Admin email/username and password
   RM email/username and password
   Employee email/username and password

3. Does the login use email or username?
   The API docs show email but our app uses username

4. What format does the login response return?

We need: token and user object with role

5. Is CORS enabled on the server?
   We need CORS allowed for http://localhost:4201

6. Can you share a Postman collection?
   This helps us test each endpoint

7. What is the response format?
   Example: { data: [...] } or { result: [...] }

Step by Step API Connection Process

STEP 1 — Update Environment File

Open this file:

E:\Attendance App\attendance-frontend\src\environments\environment.ts

Change:

```
export const environment = {
  production: false,
  apiUrl: 'REPLACE_WITH_YOUR_API_URL'
};
```

To your lead's actual URL:

```
export const environment = {
  production: false,
  apiUrl: 'http://192.168.1.100/api'
};
```

Save the file.

STEP 2 — Test the API with Postman First

Before touching Angular code test the API manually.

Test Login:

Method: POST
URL: http://your-api-url/auth/login
Body: {
  "email": "admin@company.com",
  "password": "admin123"
}
Expected response: {
  "data": {
    "token": "abc123...",
    "user": {
      "id": 1,
      "name": "Admin",
      "role": { "slug": "master_admin" }
    }
  }
}

Test Get Employees:

Method: GET
URL: http://your-api-url/employees
Headers: Authorization: Bearer YOUR_TOKEN
Expected response: {
  "data": [...]
}

If Postman works then Angular connection will work.

STEP 3 — Give This Prompt to Antigravity

Connect the Angular frontend at
E:\Attendance App\attendance-frontend
to the real PHP REST API.

The API base URL is already set in
src/environments/environment.ts

Make sure app.config.ts has HttpClient:

import { provideHttpClient, withInterceptors }

```
  from '@angular/common/http';
import { authInterceptor }
  from './core/interceptors/auth.interceptor';

providers: [
  provideRouter(routes),
  provideHttpClient(
    withInterceptors([authInterceptor])
  )
]
```

Update auth.interceptor.ts:

```
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { Router } from '@angular/router';
import { catchError, throwError } from 'rxjs';

export const authInterceptor: HttpInterceptorFn =
  (req, next) => {
    const router = inject(Router);
    const token = localStorage.getItem('token');

    const authReq = token ? req.clone({
      headers: req.headers
        .set('Authorization', 'Bearer ' + token)
        .set('Accept', 'application/json')
    }) : req.clone({
      headers: req.headers
        .set('Accept', 'application/json')
    });

    return next(authReq).pipe(
      catchError((error) => {
        if (error.status === 401) {
          localStorage.removeItem('token');
          localStorage.removeItem('user');
          router.navigate(['/login']);
        }
        return throwError(() => error);
      })
    );
  };
```

Update auth.service.ts:

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Router } from '@angular/router';
import { Observable, tap } from 'rxjs';
import { environment } from
  '../../../environments/environment';

@Injectable({ providedIn: 'root' })
export class AuthService {
  private http = inject(HttpClient);
  private router = inject(Router);
  private apiUrl = environment.apiUrl;

  login(email: string,
    password: string): Observable<any> {
    return this.http.post(
      `${this.apiUrl}/auth/login`,
      { email, password }
    ).pipe(tap((res: any) => {
      const token = res.data?.token || res.token;
      const user = res.data?.user || res.user;
      if (token) localStorage.setItem('token', token);
      if (user) localStorage.setItem('user',
        JSON.stringify(user));
    }));
  }

  loginWithUsername(username: string,
    password: string): Observable<any> {
    return this.login(username, password);
  }

  logout(): void {
    localStorage.removeItem('token');
    localStorage.removeItem('user');
    this.router.navigate(['/login']);
  }

  getCurrentUser(): any {
    const u = localStorage.getItem('user');
    return u ? JSON.parse(u) : null;
  }
```

```
  getUser(): any { return this.getCurrentUser(); }

  getToken(): string | null {
    return localStorage.getItem('token');
  }

  isLoggedIn(): boolean { return !!this.getToken(); }

  getRole(): string {
    return this.getCurrentUser()?.role?.slug || '';
  }

  getUserRole(): string { return this.getRole(); }

  clearStorage(): void {
    localStorage.removeItem('token');
    localStorage.removeItem('user');
  }
}
```

Update company.service.ts:

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { environment } from
  '../../../environments/environment';

@Injectable({ providedIn: 'root' })
export class CompanyService {
  private http = inject(HttpClient);
  private apiUrl = environment.apiUrl;

  getAll(): Observable<any> {
    return this.http.get(
      `${this.apiUrl}/companies`);
  }
  getById(id: number): Observable<any> {
    return this.http.get(
      `${this.apiUrl}/companies/${id}`);
  }
  create(data: any): Observable<any> {
    return this.http.post(
```

```
    `${this.apiUrl}/companies`, data);
  }
  update(id: number, data: any): Observable<any> {
    return this.http.put(
      `${this.apiUrl}/companies/${id}`, data);
  }
  getEmployees(id: number): Observable<any> {
    return this.http.get(
      `${this.apiUrl}/companies/${id}/employees`);
  }
}
```

Update employee.service.ts:

```
import { Injectable, inject } from '@angular/core';
import { HttpClient, HttpParams }
  from '@angular/common/http';
import { Observable } from 'rxjs';
import { environment } from
  '../../../environments/environment';

@Injectable({ providedIn: 'root' })
export class EmployeeService {
  private http = inject(HttpClient);
  private apiUrl = environment.apiUrl;

  getAll(params?: any): Observable<any> {
    let p = new HttpParams();
    if (params) {
      Object.keys(params).forEach(k => {
        if (params[k] != null)
          p = p.set(k, params[k]);
      });
    }
    return this.http.get(
      `${this.apiUrl}/employees`, { params: p });
  }
  getAllEmployees(params?: any): Observable<any> {
    return this.getAll(params);
  }
  getById(id: number): Observable<any> {
    return this.http.get(
      `${this.apiUrl}/employees/${id}`);
  }
```

```
  create(data: any): Observable<any> {
    return this.http.post(
      `${this.apiUrl}/employees`, data);
  }
  createEmployee(data: any): Observable<any> {
    return this.create(data);
  }
  update(id: number, data: any): Observable<any> {
    return this.http.put(
      `${this.apiUrl}/employees/${id}`, data);
  }
  updateEmployee(id: number,
    data: any): Observable<any> {
    return this.update(id, data);
  }
  delete(id: number): Observable<any> {
    return this.http.delete(
      `${this.apiUrl}/employees/${id}`);
  }
  deleteEmployee(id: number): Observable<any> {
    return this.delete(id);
  }
  getReportManagers(): Observable<any> {
    return this.getAll({ role: 'rm' });
  }
  getManagers(): Observable<any> {
    return this.getAll({ role: 'manager' });
  }
}
```

Update leave.service.ts:

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { environment } from
  '../../../environments/environment';

@Injectable({ providedIn: 'root' })
export class LeaveService {
  private http = inject(HttpClient);
  private apiUrl = environment.apiUrl;

  getTypes(): Observable<any> {
```

```
    return this.http.get(
      `${this.apiUrl}/leave-types`);
  }
  getAll(params?: any): Observable<any> {
    return this.http.get(
      `${this.apiUrl}/leaves`, { params });
  }
  getMyLeaves(): Observable<any> {
    return this.getAll();
  }
  getById(id: number): Observable<any> {
    return this.http.get(
      `${this.apiUrl}/leaves/${id}`);
  }
  apply(data: any): Observable<any> {
    return this.http.post(
      `${this.apiUrl}/leaves`, data);
  }
  action(id: number, action: string,
    comment: string): Observable<any> {
    return this.http.put(
      `${this.apiUrl}/leaves/${id}/action`,
      { action, comment });
  }
  cancel(id: number): Observable<any> {
    return this.http.put(
      `${this.apiUrl}/leaves/${id}/cancel`, {});
  }
  getSummary(): Observable<any> {
    return this.http.get(
      `${this.apiUrl}/leaves/summary`);
  }
  escalate(id: number,
    fromRole: string): Observable<any> {
    return this.http.put(
      `${this.apiUrl}/leaves/${id}/action`,
      { action: 'escalate',
        from_role: fromRole });
  }
}
```

Update wfh.service.ts:

```
import { Injectable, inject } from '@angular/core';
```

```
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { environment } from
  '../../../environments/environment';

@Injectable({ providedIn: 'root' })
export class WfhService {
  private http = inject(HttpClient);
  private apiUrl = environment.apiUrl;

  getAll(params?: any): Observable<any> {
    return this.http.get(
      `${this.apiUrl}/leaves`,
      { params: { ...params, type: 'wfh' } });
  }
  apply(data: any): Observable<any> {
    return this.http.post(
      `${this.apiUrl}/leaves`,
      { ...data, type: 'wfh' });
  }
  action(id: number, action: string,
    comment: string): Observable<any> {
    return this.http.put(
      `${this.apiUrl}/leaves/${id}/action`,
      { action, comment });
  }
  cancel(id: number): Observable<any> {
    return this.http.put(
      `${this.apiUrl}/leaves/${id}/cancel`, {});
  }
}
```

Update group.service.ts:

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { environment } from
  '../../../environments/environment';

@Injectable({ providedIn: 'root' })
export class GroupService {
  private http = inject(HttpClient);
  private apiUrl = environment.apiUrl;
```

```
  getAll(): Observable<any> {
    return this.http.get(`${this.apiUrl}/groups`);
  }
  getById(id: number): Observable<any> {
    return this.http.get(
      `${this.apiUrl}/groups/${id}`);
  }
  create(data: any): Observable<any> {
    return this.http.post(
      `${this.apiUrl}/groups`, data);
  }
  delete(id: number): Observable<any> {
    return this.http.delete(
      `${this.apiUrl}/groups/${id}`);
  }
  addMembers(id: number,
    userIds: number[]): Observable<any> {
    return this.http.post(
      `${this.apiUrl}/groups/${id}/members`,
      { user_ids: userIds });
  }
  removeMembers(id: number,
    userIds: number[]): Observable<any> {
    return this.http.delete(
      `${this.apiUrl}/groups/${id}/members`,
      { body: { user_ids: userIds } });
  }
}
```

After all service updates run:
ng build
Fix all errors.
Then run:
ng serve --port 4201

STEP 4 — Handle Different Response Formats

If API returns data differently update each service getAll method.

If API returns list directly:

```
[{ "id": 1, "name": "..." }]
```

Update component to handle:

```
this.companies = res.data || res;
```

This already works because we use `res.data || res`

If API returns with different key:

```
{ "companies": [...] }
```

Update service:

```
getAll() {
  return this.http.get(
    `${this.apiUrl}/companies`
  ).pipe(map((res: any) => ({
    data: res.companies || res.data || res
  })));
}
```

STEP 5 — Fix CORS if Needed

If you see this error in browser console:

```
Access to XMLHttpRequest at 'http://...' has been
blocked by CORS policy
```

Option A — Ask Lead to Fix Backend

Tell your lead:

```
Please add these headers to the PHP API response:
Access-Control-Allow-Origin: http://localhost:4201
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Authorization
```

Option B — Use Angular Proxy

Create file at project root:

```
// proxy.conf.json
{
  "/api": {
    "target": "http://your-api-url.com",
    "changeOrigin": true,
    "secure": false,
    "logLevel": "debug"
  }
}
```

Update environment.ts:

apiUrl: '/api'

Run with proxy:

ng serve --port 4201 --proxy-config proxy.conf.json

STEP 6 — Test After Connection

Test in this exact order:

Test 1 — Login

- Open http://localhost:4201
- Enter credentials from lead
- Should redirect to correct dashboard

Test 2 — Companies

- Login as admin
- Go to Companies page
- Should show real companies from database

Test 3 — Employees

- Go to Employees page
- Should show real employees from database

Test 4 — Apply Leave

- Login as employee
- Apply a leave

- Check in admin panel if it appears

Test 5 — Approve Leave

- Login as RM
- Go to Team Requests
- Approve a leave
- Check status changed

Test 6 — Full Flow

- Employee applies leave
- RM approves or escalates
- Manager approves or escalates
- Admin approves

STEP 7 — Common Errors and Fixes

Error	Cause	Fix
401 Unauthorized	Token expired or wrong	Re-login
403 Forbidden	Wrong role for endpoint	Check user role
404 Not Found	Wrong API URL	Check environment.ts URL
500 Server Error	Backend bug	Tell lead to check server logs
CORS Error	Backend not allowing frontend	Use proxy or ask lead to fix
Empty data	Response format different	Update service to handle format

---

SECTION 11 — QUICK REFERENCE

Run Commands

```
# Start app
cd "E:\Attendance App\attendance-frontend"
ng serve --port 4201

# Clear cache
Remove-Item -Recurse -Force .angular
ng serve --port 4201
```

```
# Install packages
npm install

# Build
ng build
```

API Connection Checklist

□ Get API URL from lead
□ Update environment.ts with URL
□ Give Antigravity prompt to switch services
□ Test login with real credentials
□ Test each page loads real data
□ Fix any CORS errors
□ Fix any response format differences
□ Test complete leave approval flow
□ Show to lead for review

This is the complete documentation of everything we built and exactly how to connect the real API when your lead provides it. Save this document so you have it ready when your lead gives you the API URL.